INFOHUB

PROJECT SYNOPSIS OF MINI PROJECT

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

Roll No. & Name of Students:

2501320100296 – Suhani Srivastava
2501320100289 – Sonit Mehta
2501320100285 – Shubham Kumar
2501320100304 – Swapnil Rai

SUPERVISOR / GUIDE:

Ms. Vaishali Sisaudia

Greater Noida Institute of Technology, Greater Noida

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

2026–27

INDEX

1. Introduction

2. Literature Review

3. Objectives

4. Methodology / Planning of Work

5. References

NOTATIONS / NOMENCLATURE

UI – User Interface

UX – User Experience

HTML5 – HyperText Markup Language revision 5

CSS3 – Cascading Style Sheets level 3

JS – JavaScript programming language

Flask – Python-based web application framework used for the planned backend

SQLite – Lightweight relational database planned for application data

RBAC – Role-Based Access Control framework

1. INTRODUCTION

Information sharing is a critical element in the daily operation of academic and professional institutions. However, individuals frequently encounter difficulties keeping track of important announcements, schedules, placement updates, and learning resources. InfoHub is proposed as a centralized Information Management and Aggregation System designed to streamline how critical updates are discovered, organized, and accessed within a single repository.

The platform is designed primarily for college students, faculty members, and administrators who manage recurring notices and documents. Currently, critical institutional data is frequently fragmented across separate communication channels like instant messaging groups, distinct portal links, and physical bulletin boards. This creates a highly disconnected user experience, often resulting in missed deadlines or delayed awareness of vital information.

InfoHub reorganizes this experience around a unified pipeline: "Aggregate → Categorize → Search → Retrieve". The central feature of the application is a master dashboard displaying verified notices categorized by urgency and domain (e.g., Academics, Placements, Events). Administrators have exclusive privileges to publish and tag updates, while standard users can utilize search filters to locate specific files instantly. The dashboard serves as an interactive starting point where individuals can quickly view current updates and track relevant resources.

1.1 Problem Statement

Users exploring institutional or group updates face a fragmented landscape filled with scattered communication channels and unindexed information. Existing methods rely heavily on decentralized messaging threads where critical data is buried quickly over time. This forces the user to browse through multiple platforms to locate a single relevant link or document, leading to significant time loss and communication gaps.

1.2 Proposed Solution

InfoHub will provide secure, role-based personal accounts that open to a comprehensive information dashboard. Users can read categorized announcements, perform keyword searches, filter updates by date or department, and bookmark essential resources. The initial release is intentionally focused on a foundational feature set to ensure system stability and performance before integrating complex automated scaling tools.

1.3 Technology Used

The planned application architecture relies on standard, lightweight web technologies to guarantee efficiency:

- **Frontend:** HTML5, CSS3, and JavaScript to construct a responsive interface.
- **Backend:** Flask framework to process routing, handle server responses, and manage role access.
- **Database:** SQLite database to securely store credentials, categorized notices, and system tags.
- **Versioning:** Git and GitHub to manage code revisions and facilitate team collaboration.

1.4 Major Technical Terms

- **Frontend:** The user-facing application interface built for viewing and interactions.
- **Backend:** The server-side layer managing business rules, data access, and routing requests.
- **Database:** Organized persistent storage used to track admin inputs, user profiles, and notice records.
- **Dashboard:** The main control landing page where incoming aggregated info feeds are structured for the user.
- **Role-Based Access:** A security framework restricting page actions based on the user category (Admin vs. General User).
- **User Journey:** The defined sequence of user interactions: Sign In → Dashboard → Filter Feed → Read Notice → Bookmark Resource.

1.5 Scope of the First Working Version

The primary deployment will concentrate exclusively on Secure Authorization, Admin Content Creation Tools, Categorized Dashboard Feeds, Global Search Filter, and Bookmark Collections. Advanced enhancements like automated email alerts, push notifications, AI-powered document tagging, and comment discussion boards are categorized as future scope additions.

2. LITERATURE REVIEW

The structural concept for this application stems from evaluating how institutional environments currently broadcast information to their target audiences. Standard research indicates four dominant approaches:

2.1 Standard Institutional Portals

Traditional web portals offer excellent space for static records and official documents. However, their complex design layouts can make quick navigational updates difficult for mobile users, turning simple information retrieval into an awkward task for busy students.

2.2 Instant Messaging Groups

Social applications like WhatsApp or Telegram are excellent for rapid, instantaneous broadcasting. Despite this speed, their non-indexed nature causes important files to get buried under casual conversation threads, leaving users without a clear search framework to locate historical resources.

2.3 Physical Bulletin Boards

Physical notice boards offer a centralized physical point of information but lack remote accessibility entirely. They suffer from space limitations, printing costs, and cannot support dynamic attachments, links, or real-time corrections.

2.4 Cloud File Drives

Shared drives provide massive storage and easy folder link distribution. However, they function merely as deep directories rather than dynamic, chronological announcement dashboards, requiring users to explicitly know where a file is saved beforehand.

2.5 Identified Opportunity Gap

The primary opportunity is not to replace file storage infrastructure, but to simplify everyday information navigation. The proposed platform fills this gap by delivering a structured information loop that merges rapid data publishing, neat structural filtering, and global search into one lightweight dashboard. Initial evaluations will measure navigation speed and interface clarity over long-term system modifications.

3. OBJECTIVES

The design objectives for InfoHub address the direct need for an organized, reliable, and accessible information network:

- To build a centralized, web-based platform giving users an immediate overview of all active announcements.
- To organize scattered text updates, links, and documents into neat, discoverable operational categories.
- To provide an active central dashboard displaying real-time priority alerts to stop critical communication gaps.
- To construct full text-search mechanisms allowing users to query old notices by keywords, dates, or tags.
- To implement separate account dashboards tailored strictly to administrative publishers and everyday readers.
- To deploy a bookmark saving layout so users can archive vital links for reference.
- To maintain user credentials and systemic data integrity using an integrated database architecture.
- To test the early system build with a peer focus group to capture bugs and fix layout inefficiencies.

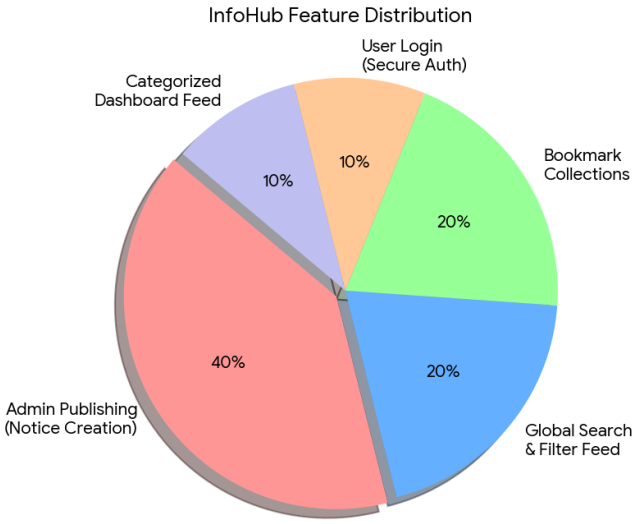
4. METHODOLOGY / PLANNING OF WORK

The development strategy follows an agile, incremental software model ensuring core database and routing modules function before secondary UI improvements are applied:

1. **Problem Definition:** Map the primary target audience requirements and plan out the baseline application page hierarchies.
2. **Feature Prioritization:** Establish the first-version scope boundaries: Registration/ Login, Publisher Panel, Category Feeds, and Keyword Search.
3. **UI / Page Design:** Wireframe responsive layouts for the main user feed and the admin document upload pages.
4. **Flask Setup:** Instantiate the primary Flask backend environments and map out foundational server routing paths.
5. **Database Integration:** Configure SQLite schemas to track relational database tables for users, logs, and notice posts.
6. **Feature Development:** Code core features including hashed security authentication, feed querying logic, and category filtering hooks.
7. **Testing:** Execute complete functional end-to-end user path checks and gather workflow usability critiques from 5–10 peer testers.
8. **Deployment:** Deliver the refined, functional web-application build for evaluation and final submission presentation.

4.1 Planned System Allocation

- **Admin Publishing Tools (40%):** Notice Creation, Priority Tagging, Relational Field Tagging.
- **User Discovery Tools (40%):** Global Key-Phrase Queries, Sectional Feeds, Archive Bookmarking.
- **Access Protocols (20%):** Multi-Role Login Verification, Relational Integrity System.



4.2 Planned Technology Stack

- **Frontend Presentational Layers:** HTML5 / CSS3 / JavaScript
- **Application Core Engine:** Flask Framework
- **Relational Storage Core:** SQLite Database
- **Version System Management:** Git / GitHub

(The comprehensive end-to-end architecture and technology stack workflow is illustrated in Figure 4.2 on the following page)

4.2 Planned Technology Stack - System Architecture Matrix

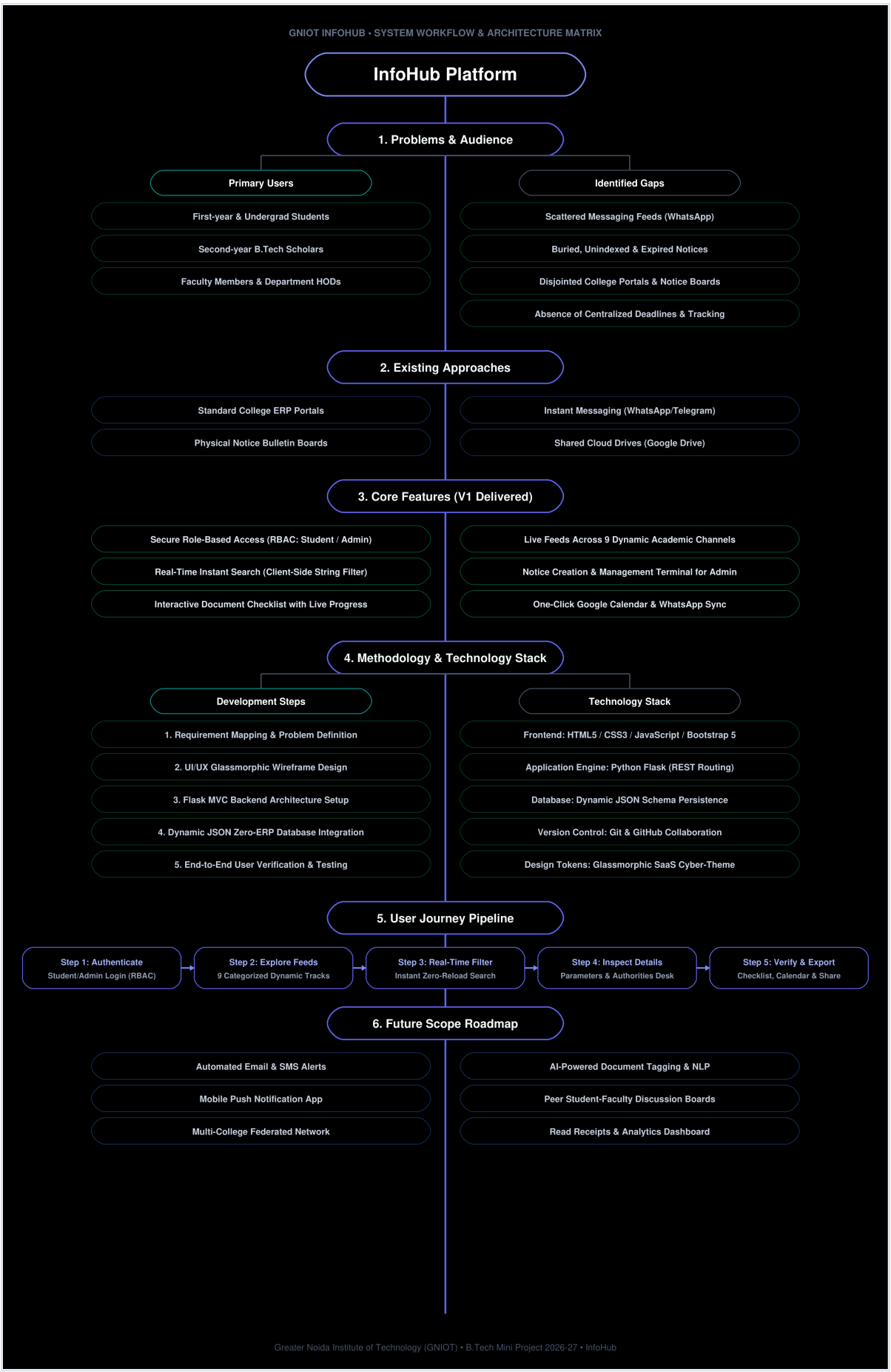


Figure 4.2: InfoHub End-to-End System Architecture & Functional Flowchart

5. REFERENCES

1. Pallets Projects, "Flask Documentation," official Flask documentation. [Online]. Available: flask.palletsprojects.com

2. SQLite Consortium, "SQLite Documentation," official SQLite documentation. [Online]. Available: sqlite.org/docs.html

3. GitHub, "GitHub Documentation," official GitHub documentation. [Online]. Available: docs.github.com

4. Bootstrap, "Bootstrap Documentation," official Bootstrap documentation. [Online]. Available: getbootstrap.com/docs